

UTN.BA

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Manual de Ingeniería Proyecto Integrador Técnicas Digitales II

Universidad Tecnológica Nacional – Facultad Regional Buenos Aires

Materia:	Técnicas Digitales II
Curso:	R4001
Cuerpo docente:	Ing. Juan Manuel Cruz Ing. Juan Carlos Paradiso Ing. Juan Ignacio Bacigalupo
Título del Proyecto:	Espectrómetro de Luz
Número de grupo:	Grupo -
Integrantes del grupo:	Franco Gabriel Méndez
	Martín Rodríguez Silvosa
Año de cursada:	2025

Índice

● Estado del Arte	3
● Principio de Funcionamiento	3
● Composición del proyecto	3
● Esquemáticos y PCB's	4
● Código programado, MCU y PC	10
● Porciones de código relevantes MCU	11
● Porciones de código relevantes PC	20
● Consumo Eléctrico:	23
● Dispositivos y periféricos empleados, detalles	23
● Tipos de comunicación empleadas	23
● Cronogramas del proyecto	24
● Costos del proyecto	25
● Desafíos	26
● Conclusiones	26
● Bibliografía	27
● Panel de firmas	27

● Estado del Arte

El espectrómetro digital está caracterizado por ser una herramienta útil en el análisis de los componentes espectrales de la luz visible, permitiendo medir la intensidad de cada longitud de onda emitida/reflejada por la fuente, calcular indirectamente temperatura de color o iluminancia dependiendo de la aplicación, o evaluar la calidad de la fuente en base a métodos de reproducción de color como los estándares CRI / TM-30.

Convierten estas señales ópticas en datos digitales a partir de métodos de procesamiento de señales a través de algoritmos y software como lo puede ser el análisis de Fourier, para luego en tiempo real obtener información cualitativa y cuantitativa de la luz medida con alta precisión.

● Principio de Funcionamiento

Nuestro proyecto utiliza principalmente un sensor acondicionado para medir pequeñas bandas de longitud de onda. La luz incide sobre las diferentes subdivisiones de longitud de onda que dispone el dispositivo y se captura una intensidad lumínica sobre cada una mediante un tiempo de integración. Esta información es almacenada en memoria volátil sobre el sensor a la espera de ser leída por el MCU. Una vez capturada y rescatada la información, se procesa mediante un algoritmo estadístico y se refleja el espectro en pantalla con un valor de intensidad por longitud de onda proporcional a la intensidad medida total.

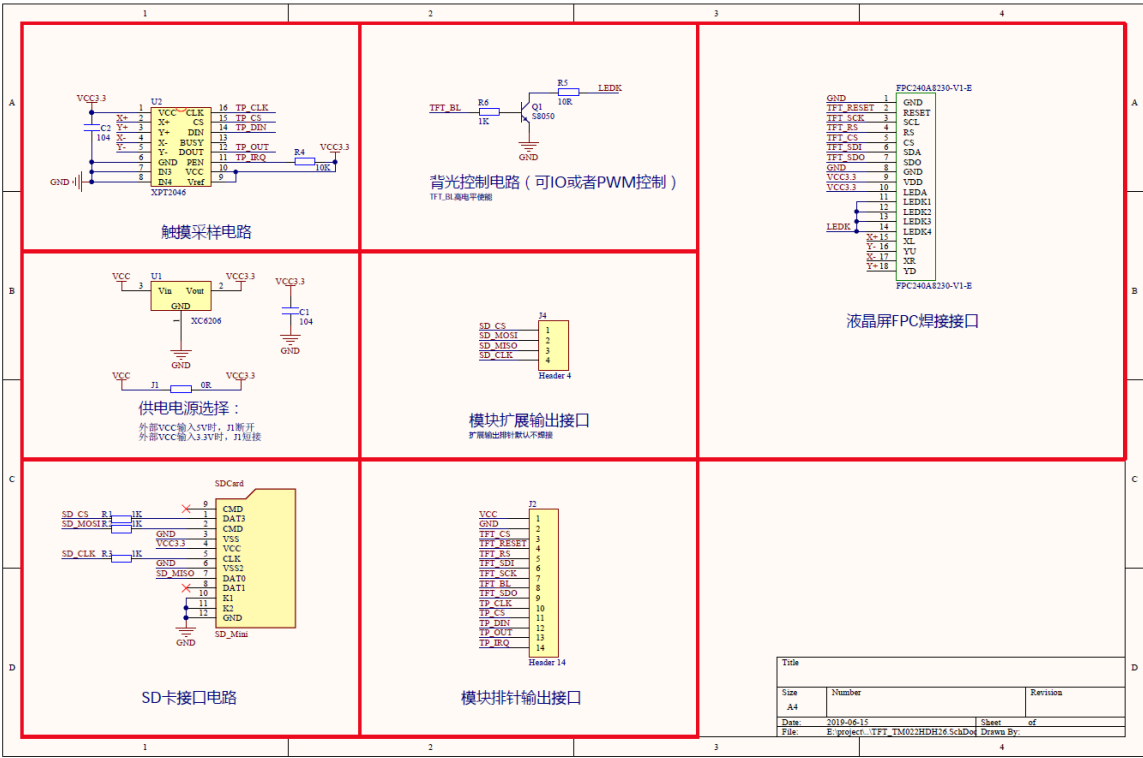
● Composición del proyecto

Para desarrollar esta tarea, decidimos utilizar los siguientes dispositivos:

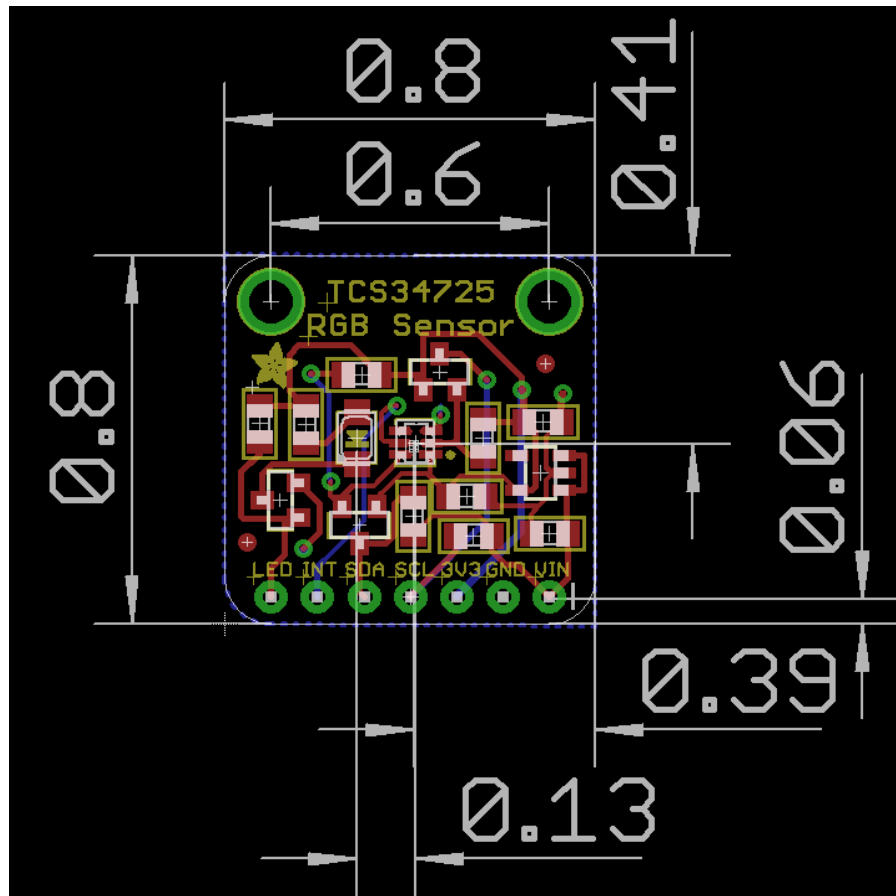
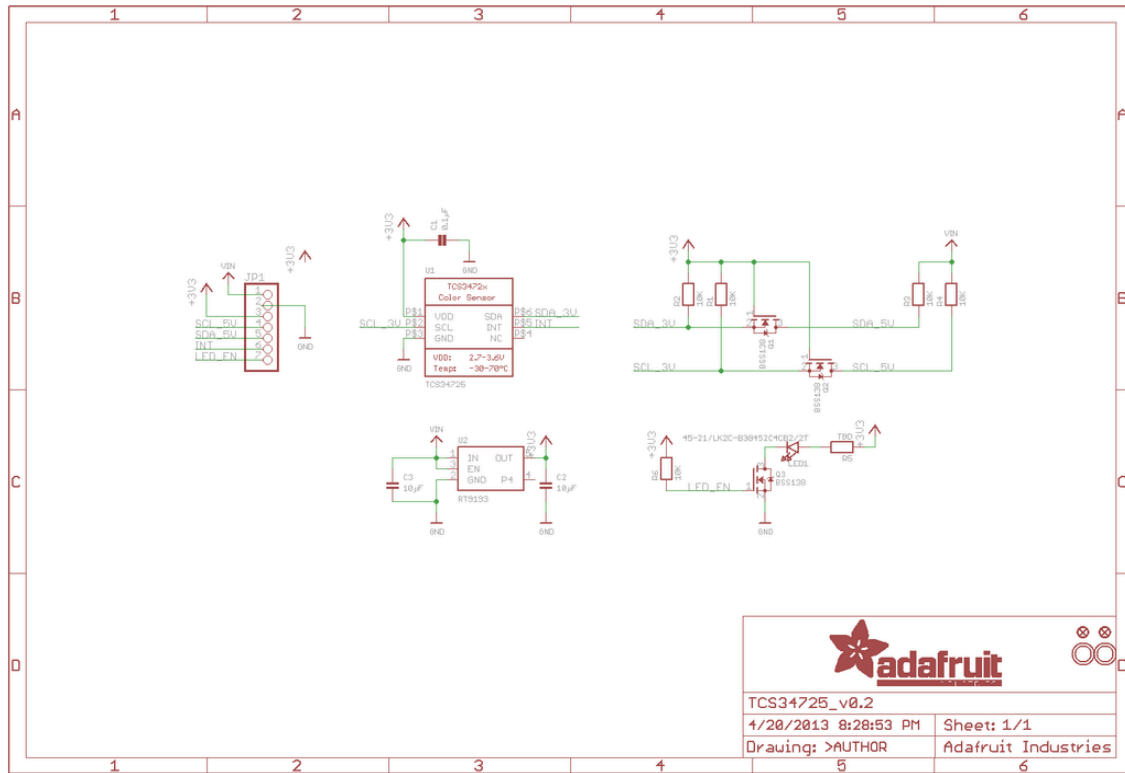
- STM32 Núcleo-F446RE como MCU principal
- Pantalla TFT ILI9341 para la visualización de la interfaz gráfica
- Memoria SD integrada dentro del ILI9341 para el almacenamiento del GUI
- Sensor TCS34725 como análisis espectral ante el fallo imprevisto del AS7262
- Sensor LDR de intensidad lumínica para la retroiluminación de la pantalla
- Pulsadores varios para el desplazamiento y selección dentro del GUI
- Baterías AA como fuente de alimentación del dispositivo portátil
- Llave selectora para el encendido y apagado del proyecto
- Conector USB para subir los datos espectrales tomados a PC con aplicación propia
- Gabinete impreso en 3D que almacena todos los dispositivos empleados

● Esquemáticos y PCB's

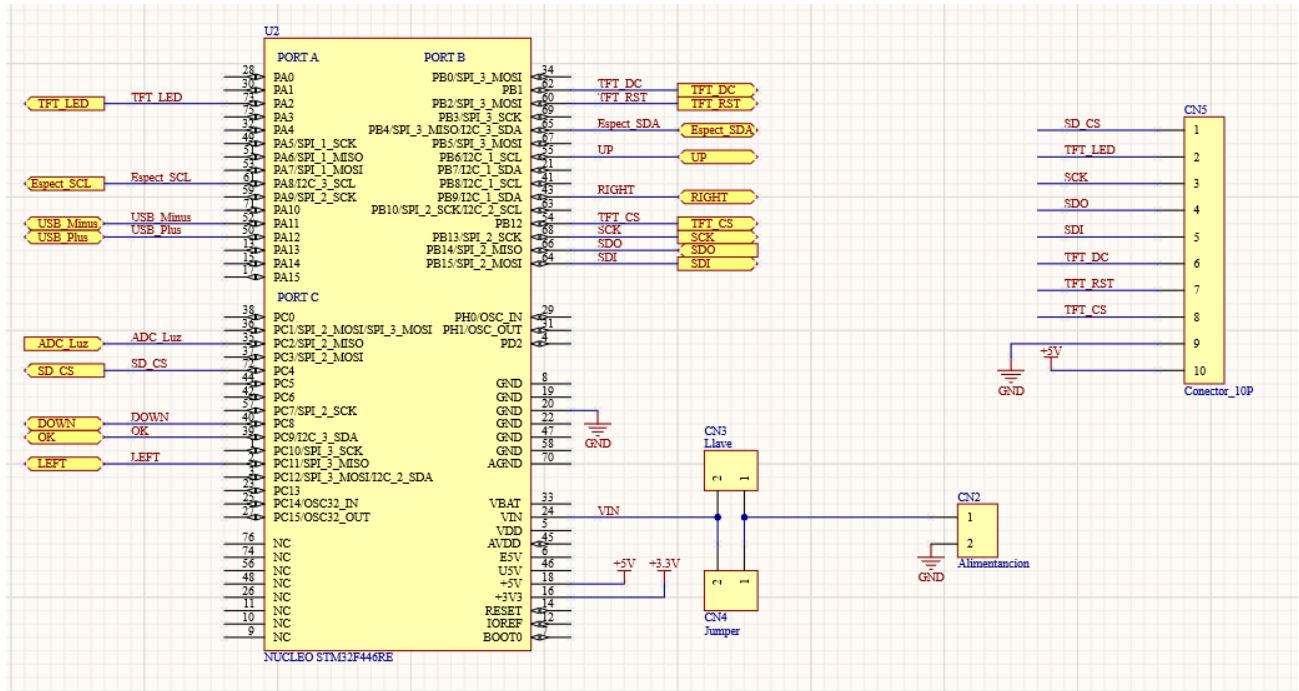
○ Pantalla TFT ILI9341 + SD:



- Sensor TCS34725:

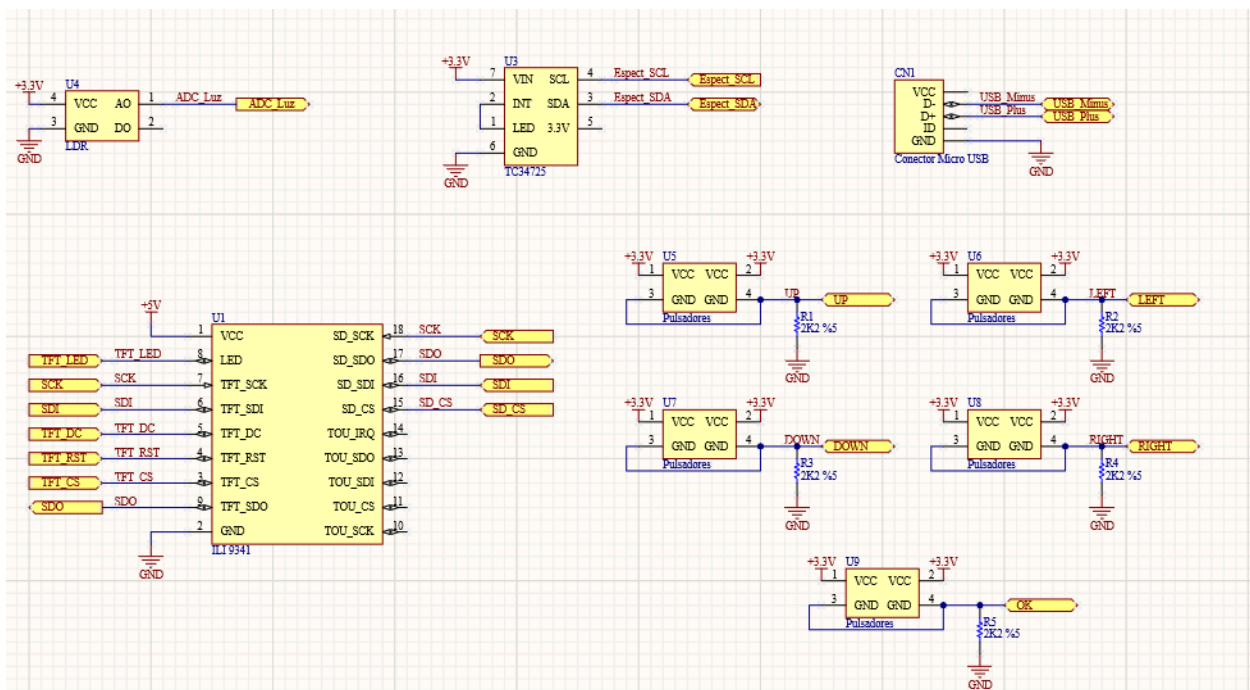


○ Núcleo:



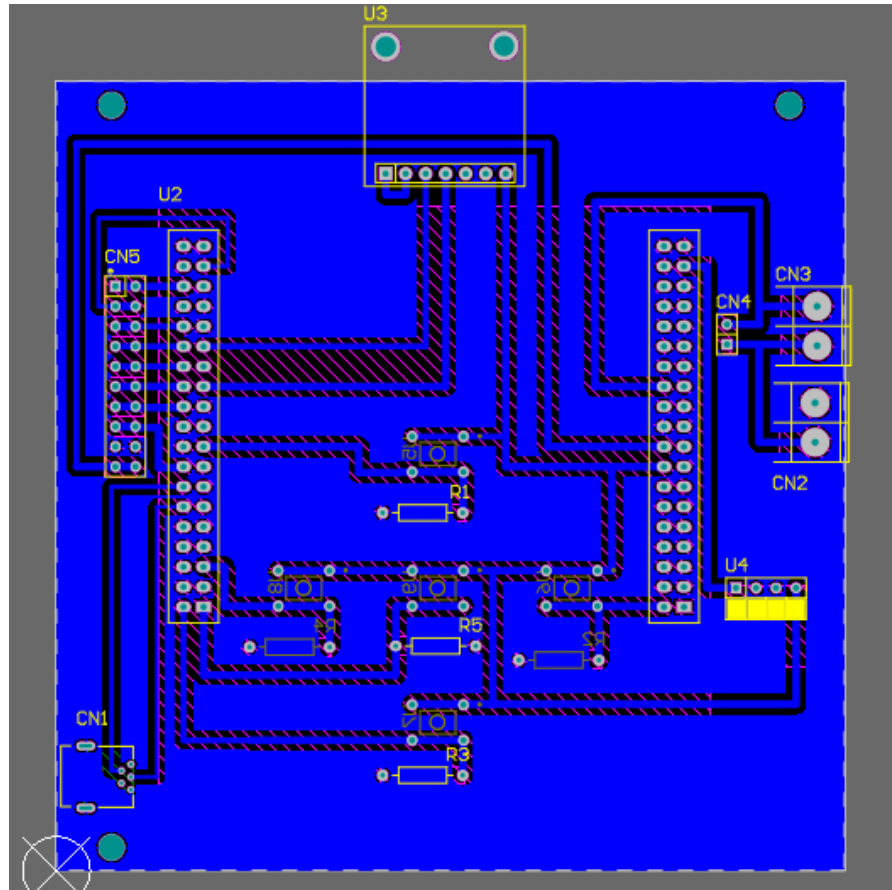
Se puede observar la placa utilizada y la designación de los pines tiene asignado la correspondiente función que llevará a cabo, ya sea comunicación de datos/clock, ADC, USB, etc.

○ Periféricos:

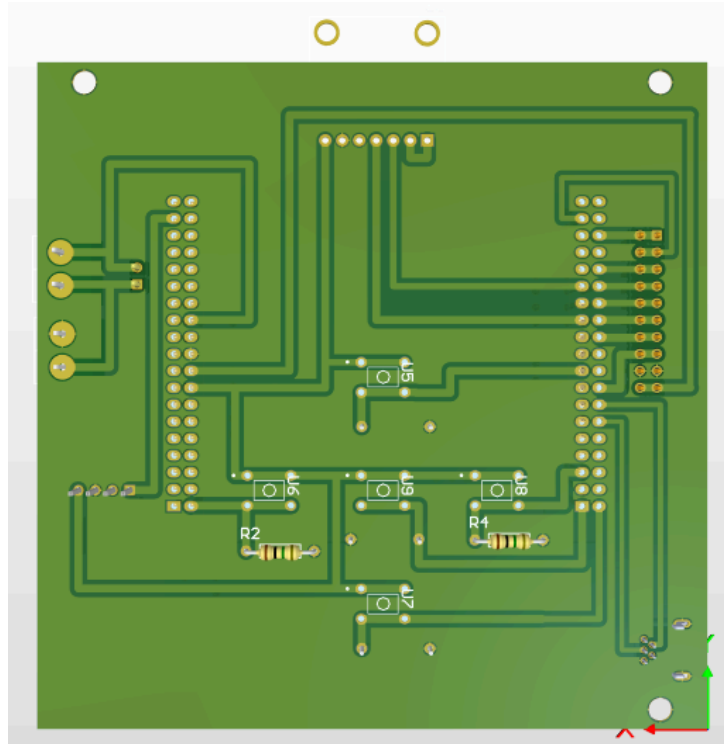


Se puede observar como fueron alimentados cada periféricos y como sus respectivas conexiones.

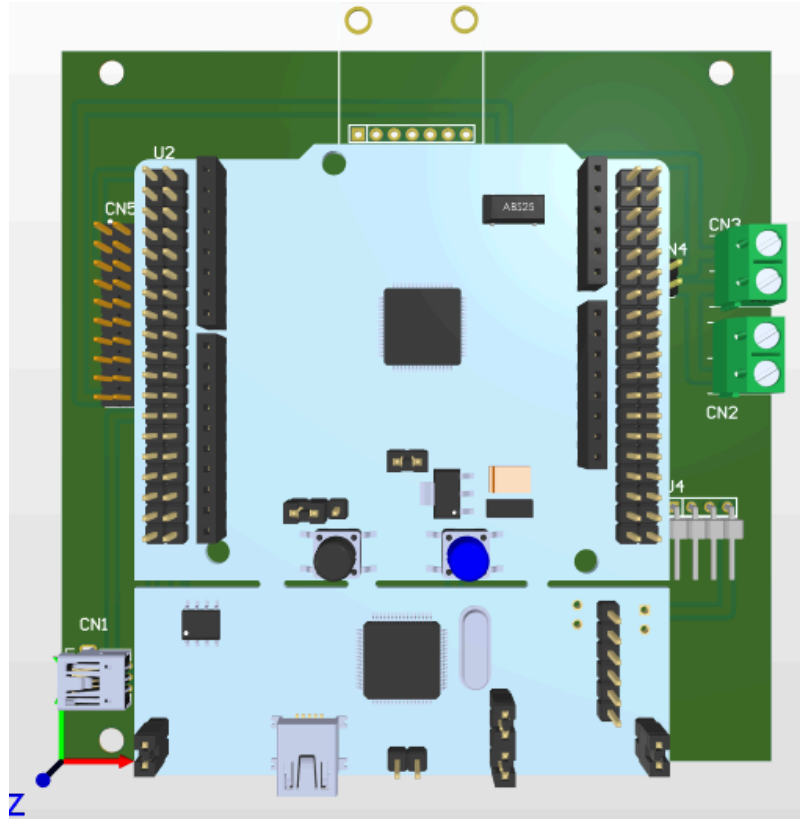
- PCB:



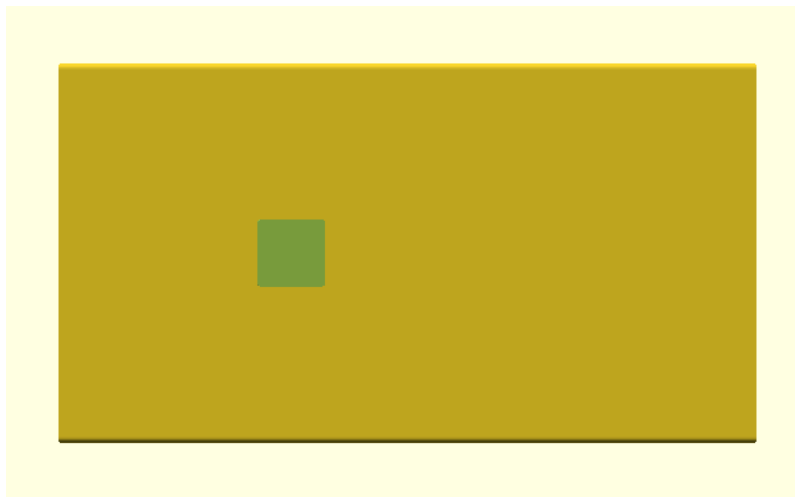
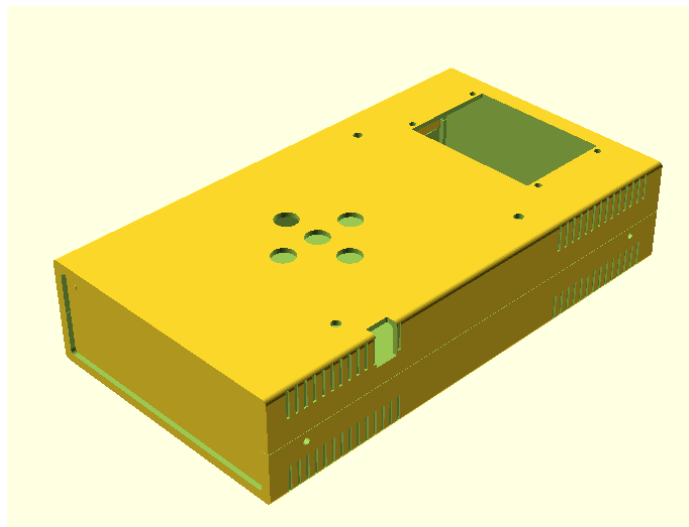
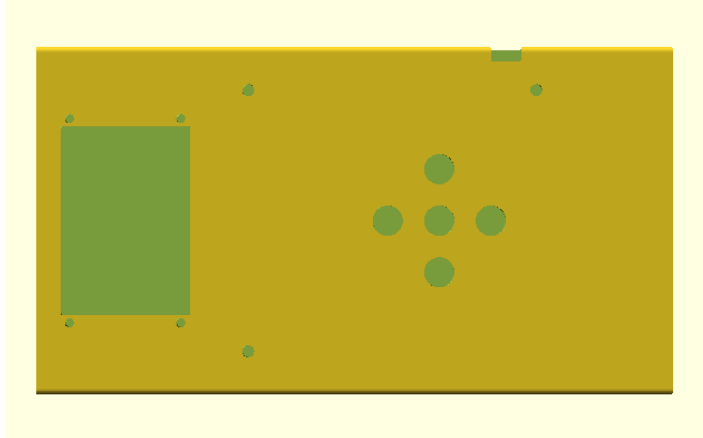
- 3D Vista Frontal:



- 3D Vista Posterior:



- Gabinete 3D:



● Código programado, MCU y PC

El desarrollo del proyecto contempló la realización de un *STM Project* dentro del [STM32CubeIDE](#) + [STM32CubeMX](#) como software oficial para programar en lenguaje C la Núcleo-F446RE que utilizamos como micro controlador principal. A su vez se realizó una aplicación de PC mediante el uso de [Qt Creator](#) programado en C++.

El código del MCU contiene las siguientes librerías utilizadas:

- Librería [HAL F4xx](#) conteniendo todo lo necesario para utilizar los periféricos
- Librería [FreeRTOS](#) conteniendo lo necesario para la implementación de un sistema operativo necesario para mantener el paralelismo de tareas
- Librería [USB Device Library](#) implementada por STM para facilitar el uso de USB
- Librerías para el manejo del ILI9341 y del TCS34725 provistas por [Adafruit](#) y terceros, que fueron modificadas para su uso sobre STM
- Librería SD creada a partir de documentación detallada en clase

El código de PC contiene las siguientes librerías destacadas dentro del entorno Qt:

- Librería [QFile](#) para el manejo de archivos
- Librería [QDateTime](#) para la implementación de guardado según fecha de uso
- Librería [libusb](#) para el control y manejo de la comunicación por USB

- Porciones de código relevantes MCU

Tarea T_LDR(). Tarea de baja prioridad encargada de leer el valor analógico entregado por el módulo LDR. Toma un número de muestras con el ADC que luego se promedia y en base a eso se genera un pulso PWM con un DUTY mínimo y máximo que se inyecta a la pantalla para el manejo de la retroalimentación. A mayor intensidad lumínica ambiente, mayor brillo sobre la pantalla y viceversa.

```

262
263 void T_LDR(void* args)
264 {
265     const float DUTY_MIN = 0.05f; // duty mínimo arbitrario
266     const float DUTY_MAX = 1.0f;
267     const uint16_t PWM_PERIOD00 = 1000; // ARR del tim4
268     const uint32_t delay = 100; // 100ms de espera entre lecturas
269     uint32_t ADC_valor = 0;
270     float valor_normalizado = 0.0f;
271     float duty[10] = {[0 ... 9]=0.0f};
272     float duty_promedio = 0.0f;
273     uint32_t PWM_valor = 0;
274     uint32_t duty_index = 0;
275     const uint32_t duty_amount_max = 10;
276
277     while(1)
278     {
279         HAL_ADC_Start(&hadc1); // empiezo lectura adc
280
281         // defino un timeout arbitrario y espero a que el adc esté ready:
282         if(HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
283         {
284             ADC_valor = HAL_ADC_GetValue(&hadc1); // 12 bits, 0 a 4095
285
286             HAL_ADC_Stop(&hadc1);
287
288             // tomo el porcentaje según mi valor máximo del duty:
289             valor_normalizado = 1.0f - ((float)ADC_valor / 4095.0f);
290
291             // igualo al duty teniendo en cuenta el valor mínimo:
292             duty[duty_index] = DUTY_MIN + valor_normalizado * (DUTY_MAX - DUTY_MIN);
293
294             // limito a máximos y mínimos:
295             if(duty[duty_index] < DUTY_MIN){ duty[duty_index] = DUTY_MIN; }
296             if(duty[duty_index] > DUTY_MAX){ duty[duty_index] = DUTY_MAX; }
297
298             duty_promedio = 0.0f;
299
300             for(uint32_t i = 0; i < duty_amount_max; i++)
301             {
302                 duty_promedio += duty[i];
303             }
304             duty_promedio /= duty_amount_max;
305
306             if(duty_promedio < DUTY_MIN){ duty_promedio = DUTY_MIN; }
307             if(duty_promedio > DUTY_MAX){ duty_promedio = DUTY_MAX; }
308
309             PWM_valor = (uint32_t)(duty_promedio * PWM_PERIOD00); // insertamos para el ARR
310
311             __HAL_TIM_SET_COMPARE(&tim3, TIM_CHANNEL_2, PWM_valor);
312
313             duty_index++;
314             duty_index %= duty_amount_max;
315             vTaskDelay(delay); // Espero a la siguiente toma de valores para no saturar.
316         }
317     }
318 }

```

Tarea T_MdE(). Tarea de alta prioridad encargada del manejo de instancias dentro del menú. Es la tarea principal y por tanto una tarea larga y compleja, encargada de coordinar las acciones que se tienen que realizar para el correcto funcionamiento del dispositivo.

```

1  "main.c" X
732n void T_MDE(void* arg)
733 {
734     static uint32_t timeout = 0;
735     static uint8_t MDE = 0;
736     static userInteraction_t inputUserReceived;
737     static menuPosition_t menu_previoustage = UP_LEFT;
738     static menuPosition_t menu_poststage = UP_LEFT;
739     static menuPosition_t ESPECTRO_previoustage = DOWN_LEFT;
740     static menuPosition_t ESPECTRO_poststage = DOWN_LEFT;
741     static menuPosition_t OPCIONES_previoustage = UP_LEFT;
742     static menuPosition_t OPCIONES_poststage = UP_LEFT;
743     static IL19341_action_t IL19341_action{
744         .imageSelected = LOGO,
745         .petition = 0,
746         .rotation = 0
747     };
748     static T_S0_action_t S0_action{
749         .block = 0,
750         .imageSelected = LOGO,
751         .petition = 0
752     };
753
754     while(1)
755     {
756         switch(MDE)
757         {
758             case 0:
759                 IL19341_action.petition = 0; // set screen BLACK
760                 IL19341_action.rotation = 2;
761                 xQueueSendToBack(Q(IL19341, &IL19341_action, portMAX_DELAY);
762                 MDE = 1;
763                 break;
764             case 1:
765                 // Espero que esté libre la pantalla:
766                 xSemaphoreTake(S_IL19341, portMAX_DELAY);
767
768                 IL19341_action.petition = 1; // load whole image
769                 IL19341_action.rotation = 3;
770                 IL19341_action.imageSelected = LOGO;
771                 S0_action.imageSelected = LOGO;
772                 S0_action.petition = 1;
773                 xQueueSendToBack(Q(S0, S0_action, portMAX_DELAY);
774                 xQueueSendToBack(Q(IL19341, &IL19341_action, portMAX_DELAY);
775                 vTaskDelay(3000);
776                 MDE = 2;
777                 break;
778             case 2:
779                 // Espero que esté libre la pantalla:
780                 xSemaphoreTake(S_IL19341, portMAX_DELAY);
781                 // Espero que esté sin usar la S0:
782                 vSemaphoreTake(S0, portMAX_DELAY);
783
784                 MDE = 2; // no llegó el dato o fue erroneo, vuelvo al menu.
785                 if(flag != 1)
786                 {
787                     MDE = 2; // no llegó el dato o fue erroneo, vuelvo al menu.
788                 }
789                 else
790                 {
791                     MDE = 14; // llegó la petición desde PC
792                 }
793                 break;
794             case 14:
795                 // llegó petición, procedo a mandar los datos:
796                 IL19341_action.petition = 13; // muestro que se mandan los datos.
797                 IL19341_action.rotation = 2;
798                 // Espero que esté libre la pantalla:
799                 xSemaphoreTake(S_IL19341, portMAX_DELAY);
800                 xQueueSendToBack(Q(IL19341, &IL19341_action, portMAX_DELAY);
801                 int8_t error = USB_task();
802                 if(error == 0)
803                     MDE = 15;
804                 if(error == -1)
805                     MDE = 16;
806                 break;
807             case 15:
808                 // muestro que se terminaron de enviar los datos y vuelvo al menu:
809                 IL19341_action.petition = 14; // muestro que se terminó.
810                 IL19341_action.rotation = 2;
811                 // Espero que esté libre la pantalla:
812                 xSemaphoreTake(S_IL19341, portMAX_DELAY);
813                 xQueueSendToBack(Q(IL19341, &IL19341_action, portMAX_DELAY);
814                 vTaskDelay(2000);
815                 MDE = 2;
816                 break;
817             case 16:
818                 // muestro que hubo un error en la comunicación usb y vuelvo al menu:
819                 IL19341_action.petition = 15; // muestro el error.
820                 IL19341_action.rotation = 2;
821                 // Espero que esté libre la pantalla:
822                 xSemaphoreTake(S_IL19341, portMAX_DELAY);
823                 xQueueSendToBack(Q(IL19341, &IL19341_action, portMAX_DELAY);
824                 vTaskDelay(2000);
825                 MDE = 2;
826                 break;
827             default:
828                 break;
829         }
830     }
831 }

```

[Haga click aquí para volver al índice](#)

Tarea T_ILI9341(). Tarea de alta prioridad encargada del manejo exclusivo de la pantalla. Dependiendo de las peticiones que le lleguen, determina lo que tiene que mostrar en pantalla. Su tiempo de ejecución se confina a únicamente cuando le llega 1 petición.

```
402 void T_ILI9341(void* args)
403 {
404     static T_ILI9341_action_t userDisplayPetition;
405     static uint32_t bufferILI9341Index = 0;
406     static uint32_t blockILI9341Index = 0;
407     static bool ILI9341_imagePrinting = false;
408     static bool isFirstBlock = false;
409     static uint32_t width = 0;
410     static uint32_t height = 0;
411     static char stringTimeout[] = "TIMEOUT: 5 SEG ..";
412     stringTimeout[9] = 'S';
413     static char integrationTimeText[10] = "2.5 ms";
414     static char gainText[10] = "x4";
415     static uint8_t integrationTimeTextIndex = 0;
416     const uint8_t max_integrationTimeTextIndex = 19;
417     static uint8_t integrationTimeActive = TCS34725_INTEGRATIONTIME_2_4MS;
418     static uint8_t integrationTimeSelected = TCS34725_INTEGRATIONTIME_2_4MS;
419     static uint8_t gainTextIndex = 0;
420     const uint8_t max_gainTextIndex = 4;
421     static uint8_t gainActive = TCS34725_GAIN_4X;
422     static uint8_t gainSelected = TCS34725_GAIN_4X;
423     while(1)
424     {
425         // Espero que me llamen para poner algo en pantalla, lo que sea:
426         xQueueReceive(Q_ILI9341, &userDisplayPetition, portMAX_DELAY);
427         if(userDisplayPetition.rotation == 3 || userDisplayPetition.rotation == 1)
428         {
429             width = 240;
430             height = 320;
431         }
432         else
433         {
434             width = 320;
435             height = 240;
436         }
437         // analizo qué me pidieron:
438         switch(userDisplayPetition.petition)
439         {
440             case 0:
441                 // print BLACK screen:
442                 ILI9341_setRotation(2);
443                 ILI9341_fill_DMA(COLOR_BLACK, width, height, bufferRx);
444                 break;
445             case 1:
446                 // print uhla image:
447                 break;
448             case 2:
449                 // cambio al próximo texto de gain:
450                 switch(gainTextIndex)
451                 {
452                     case 0:
453                         strcpy(gainText, "x16");
454                         gainSelected = TCS34725_GAIN_16X;
455                         break;
456                     case 1:
457                         strcpy(gainText, "x64");
458                         gainSelected = TCS34725_GAIN_64X;
459                         break;
460                     case 2:
461                         strcpy(gainText, "x1");
462                         gainSelected = TCS34725_GAIN_1X;
463                         break;
464                     case 3:
465                         strcpy(gainText, "x4");
466                         gainSelected = TCS34725_GAIN_4X;
467                         break;
468                     default:
469                         break;
470                 }
471                 gainTextIndex++;
472                 gainTextIndex %= max_gainTextIndex;
473                 break;
474             case 3:
475                 // apply changes:
476                 gainActive = gainSelected;
477                 integrationTimeActive = integrationTimeSelected;
478                 Adafruit_TCS34725_setGain(gainActive);
479                 Adafruit_TCS34725_setIntegrationTime(integrationTimeActive);
480                 break;
481             default:
482                 break;
483         }
484         OPCIONES_desplazamiento_print(userDisplayPetition.positionReceived, userDisplayPetition.rotation);
485         break;
486     }
487     xSemaphoreGive(S_ILI9341);
488 }
```

Tarea T_SD(). Tarea de alta prioridad encargada del manejo exclusivo de la SD. Dependiendo de las peticiones que le lleguen, determina la información que tiene que rescatar de memoria. Su tiempo de ejecución se confina a únicamente cuando le llega 1 petición.

```
320 void T_SD(void* args)
321 {
322     static T_SD_action_t userSDPetition;
323     static uint32_t bufferSDIndex = 0;
324     static uint32_t blockSDIndex = 0;
325     static bool SD_imageLoading = false;
326     static bool success = false;
327     while(1)
328     {
329         // Espero que me llamen para buscar algo, lo que sea:
330         xQueueReceive(Q_SD, &userSDPetition, portMAX_DELAY);
331         // analizo qué me pidieron:
332         switch(userSDPetition.petition)
333         {
334             case 1:
335                 // load image:
336                 // iteramos 300 bloques:
337                 // trabajará en equipo con la tarea del ILI9341:
338                 // se utilizan buffers globales y flags globales por la velocidad.
339                 SD_imageLoading = true;
340                 while(SD_imageLoading)
341                 {
342                     if((buffer_ready[bufferSDIndex] == BUFFER_LOADING))
343                     {
344                         // Espero que termine el SPI:
345                         xSemaphoreTake(S_SD_ISR, portMAX_DELAY);
346                         //volatile HAL_SPI_StateTypeDef st = hspi2.State;
347                         //volatile uint32_t err = HAL_SPI_GetError(&hspi2);
348                         SD_BSP_SPI_transmitReceive_DMA_waitReady();
349                         SD_BSP_SPI_transmitReceive_DMA_finish();
350                         buffer_ready[bufferSDIndex] = BUFFER_READY;
351                         bufferSDIndex++;
352                         bufferSDIndex %= 4;
353                         sd_wait_ready();
354                         if(blockSDIndex >= 300)
355                         {
356                             SD_imageLoading = false;
357                         }
358                     }
359                     if(SD_imageLoading == false)
360                     {
361                         continue;
362                     }
363                     if((buffer_ready[bufferSDIndex] == BUFFER_IDLE))
364                     {
365                         success = true; // state inicial para saltar el error state.
366                         do
367                         {
368                             if(!success)
369                             {
370                                 sd_wait_ready();
371                                 success = sd_read_block_DMA(userSDPetition.imageSelected + blockSDIndex);
372                             }while(!success);
373                             buffer_ready[bufferSDIndex] = BUFFER_LOADING;
374                             blockSDIndex++;
375                         }
376                         bufferSDIndex = 0;
377                         blockSDIndex = 0;
378                         break;
379                     }
380                     // rescatar solamente la parte espectro de ESPECTRO_COLOR:
381                     image_select_t image = ESPECTRO_COLOR;
382                     ESPECTRO_cargarEspectroColor(Espectro_x1, Espectro_y1, Espectro_x2, Espectro_y2);
383                     break;
384                 }
385                 default:
386                     // no me pidieron nada
387                     break;
388             }
389         }
390         xSemaphoreGive(S_SD);
391     }
392 }
```

Tareas T_{SD} + $T_{ILI9341}$, funcionamiento en conjunto: A continuación se detallarán las porciones de código que se ejecutan en la instancia de recuperación de la imagen en memoria SD y su impresión sobre la pantalla. Para llevar a cabo esta función, se tuvo que implementar un sistema de alternancia de buffers cuya escritura se hace a través de canales DMA, lo cual conllevó un desafío tecnológico para su correcta sincronización y permitió ejecutar la impresión de las diapositivas del menú principal a velocidades inalcanzables de otra manera, logrando un tiempo de impresión de la imagen total menor a 300 milisegundos.

Tarea T_{SD} , rescate de información: Después de recibir la directiva de rescatar determinada imagen de memoria de parte de la tarea principal, se predispone a iterar el respectivo rescate hasta haber cargado la totalidad de la imagen. La información en la SD está guardada y se accede por bloques de memoria de 512 bytes, y cada imagen ocupa un total de 300 bloques exactos en memoria. Se dispone de 4 buffers cada uno con su respectivo indicador de estado actual. Los buffers empiezan en un estado inactivo, lo cual le da la indicación a la tarea SD que puede sobrescribirlos con nueva información.

Lo primero que realiza la tarea es rescatar 1 bloque de memoria utilizando 1 canal DMA. Para ello ejecuta 1 función encargada de pedir la información, pero el canal DMA será el encargado de indicar que la información ya fue cargada sobre el buffer, dejando en el mientras tanto al buffer en estado LOADING para indicarle a la tarea su estado actual y que tiene que esperar. Una vez que el canal DMA libera al buffer, se indica que el estado actual de ese buffer es READY y se deja listo para su impresión en pantalla. Hecho esto, se predispone a realizar la carga del próximo bloque con el próximo buffer que esté en estado inactivo. A medida que la pantalla imprima la información rescatada, irá liberando los buffers antiguos para poder reutilizarlos.

```
main.c
335 case 1:
336     // load image;
337     // iteramos 300 bloques:
338     // trabajará en equipo con la tarea del ILI9341:
339     // se utilizan buffers globales y flags globales por la velocidad.
340     SD_imageLoading = true;
341     while(SD_imageLoading)
342     {
343         if((buffer_ready[bufferSDIndex] == BUFFER_LOADING))
344         {
345             // Espero que termine el SPI:
346             xSemaphoreTake(S_SD_ISR, portMAX_DELAY);
347
348             //volatile HAL_SPI_StateTypeDef st = hspi2.State;
349             //volatile uint32_t err = HAL_SPI_GetError(&hspi2);
350
351             SD_BSP_SPI_transmitReceive_DMA_waitReady();
352             SD_BSP_SPI_transmitReceive_DMA_finish();
353
354             buffer_ready[bufferSDIndex] = BUFFER_READY;
355             bufferSDIndex++;
356             bufferSDIndex %= 4;
357
358             sd_wait_ready();
359             if(blockSDIndex >= 300)
360             {
361                 SD_imageLoading = false;
362             }
363         }
364         if(SD_imageLoading == false)
365         {
366             continue;
367         }
368
369         if((buffer_ready[bufferSDIndex] == BUFFER_IDLE))
370         {
371             success = true; // state inicial para saltar el error state.
372             do
373             {
374                 if(!success)
375                 {
376                     sd_wait_ready();
377                 }
378                 success = sd_read_block_DMA(userSDPetition.imageSelected + blockSDIndex, dummyBufferTx, buffers[bufferSDIndex]);
379             }while(!success);
380
381             buffer_ready[bufferSDIndex] = BUFFER_LOADING;
382             blockSDIndex++;
383         }
384     }
385     bufferSDIndex = 0;
386     blockSDIndex = 0;
387     break;
```

Tarea T_ILI9341, impresión de información: Después de recibir la directiva de imprimir determinada imagen en pantalla de parte de la tarea principal, se predispone a iterar la respectiva impresión hasta haber impreso la totalidad de la imagen.

Lo primero que realiza la tarea es buscar que el primer buffer esté en estado READY. Una vez que esto suceda, procede a ejecutar una función encargada de imprimir el buffer en pantalla mediante el uso de 1 canal DMA, pero el canal DMA será el encargado de indicar que la información ya fue impresa en la pantalla, dejando en el mientras tanto al buffer en estado PRINTING para indicarle a la tarea su estado actual y que tiene que esperar. Una vez que el canal DMA libera al buffer, se indica que el estado actual de ese buffer es IDLE y se deja listo para un nuevo rescate en memoria. Hecho esto, se predispone a realizar la carga del próximo buffer que esté en estado READY sobre la pantalla. A medida que la SD rescate nueva información, irá reescribiendo los buffers antiguos para poder seguir imprimiendo.

```
*main.c X
450     break;
451     case 1:
452         // print whole image:
453         // se iterarán 300 bloques:
454         // trabajará en equipo con la tarea del SD:
455         // se utilizan buffers globales y flags globales por la velocidad.
456         ILI9341.setRotation(userDisplayPetition.rotation);
457         if(userDisplayPetition.rotation == 3 || userDisplayPetition.rotation == 1)
458         {
459             width = 240;
460             height = 320;
461         }
462         else
463         {
464             width = 320;
465             height = 240;
466         }
467         ILI9341_imagePrinting = true;
468         isFirstBlock = true;
469         while(ILI9341_imagePrinting)
470         {
471             if(buffer_ready[bufferILI9341Index] == BUFFER_PRINTING)
472             {
473                 // Espero que termine el SPI:
474                 xSemaphoreTake(S_ILI9341_ISR, portMAX_DELAY);
475                 ILI9341.printImageSector_512bytes_DMA_finish();
476
477                 buffer_ready[bufferILI9341Index] = BUFFER_IDLE;
478                 bufferILI9341Index++;
479                 bufferILI9341Index %= 4;
480                 if(blockILI9341Index >= 300)
481                 {
482                     ILI9341_imagePrinting = false;
483                 }
484             }
485             if(ILI9341_imagePrinting == false)
486             {
487                 continue;
488             }
489             if(buffer_ready[bufferILI9341Index] == BUFFER_READY)
490             {
491
492                 ILI9341.printImageSector_512bytes_DMA_start(0, 0, width, height, buffers[bufferILI9341Index], isFirstBlock);
493                 buffer_ready[bufferILI9341Index] = BUFFER_PRINTING;
494                 isFirstBlock = false;
495                 blockILI9341Index++;
496             }
497         }
498         bufferILI9341Index = 0;
499         blockILI9341Index = 0;
500         break;
501     case 2:
502         // Imprimo el default selected del Menú:
```

Liberación de los buffers, canales DMA, interrupciones del sistema: Como mencionamos antes, los canales DMA liberan los buffers de ambas tareas previas para indicar la terminación de la operación. Existen 3 canales DMA que estarán linkeados a diferentes callbacks, ya que la SD utiliza un callback de escritura y lectura mientras que la pantalla utiliza un callback de sólo escritura.

```
*main.c X
2135 }
2136
2137 void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef* hspi)
2138 {
2139     BaseType_t* xHigherPriorityTaskWoken = pdFALSE;
2140
2141     if(hspi->Instance == SPI1) // ILI9341
2142     {
2143         // descomentar para identificar errores:
2144         //volatile HAL_SPI_StateTypeDef st = hspi->State;
2145         //volatile uint32_t err = HAL_SPI_GetError(hspi);
2146         xSemaphoreGiveFromISR(S_ILI9341_ISR, xHigherPriorityTaskWoken);
2147         portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
2148     }
2149 }
2150
2151 void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
2152 {
2153     BaseType_t* xHigherPriorityTaskWoken = pdFALSE;
2154
2155     if(hspi->Instance == SPI2) // SD
2156     {
2157         // descomentar para identificar errores:
2158         //volatile HAL_SPI_StateTypeDef st = hspi->State;
2159         //volatile uint32_t err = HAL_SPI_GetError(hspi);
2160         xSemaphoreGiveFromISR(S_SD_ISR, xHigherPriorityTaskWoken);
2161         portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
2162     }
2163 }
2164
```

Tarea T_SD, rescate del espectro total: Una de las directivas que acepta la tarea SD es la de rescatar y colocar en un buffer la totalidad de los píxeles correspondientes al espectro entero con la máxima intensidad posible. Esto se realiza para que luego la pantalla pueda elegir qué píxeles utilizar dependiendo de lo que haya dado la medición espectral. El espectro está guardado como una diapositiva entera para no alterar el algoritmo de guardado de imágenes de la SD, por lo que tendremos que generar una lógica de rescate que me permita obtener exclusivamente aquellos píxeles relevantes y minimizar memoria utilizada, ya que estas imágenes ocupan mucho espacio.

Función `ESPECTRO_cargarEspectroColor()`: Aquí se confina la lógica necesaria para rescatar los píxeles relevantes para la impresión del espectro. Se realizan múltiples lecturas sobre múltiples bloques de la diapositiva en SD en busca de los píxeles pertinentes

```
*main.c X
1896 void ESPECTRO_cargarEspectroColor(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, image_select_t imageSelected)
1897 {
1898     uint16_t width; // ancho espectro 296
1899     //uint16_t height; // altura espectro 101
1900     uint32_t EspectroBufferIndex = 0;
1901     uint32_t firstPixel, firstByte, block, offset;
1902     int32_t bytesNeeded, bytesAvailable, bytesTakenFromRow;
1903     uint32_t lastBlockLoaded = 301; // valor imposible a propósito
1904
1905     width = x2 - x1 + 1;
1906     //height = y2 - y1 + 1;
1907
1908     for(uint16_t y = y1; y <= y2; y++)
1909     {
1910         // Es más fácil rescatar por fila:
1911         firstPixel = y * 320 + x1;
1912
1913         firstByte = firstPixel * 2; // uint16 vs uint8
1914         block = firstByte / 512; // bloque donde se encuentra
1915         offset = firstByte % 512; // offset desde el bloque donde se encuentra
1916         bytesNeeded = width * 2; // 296 * 2 = 592 bytes, más de 1 bloque.
1917
1918         while(bytesNeeded > 0)
1919         {
1920             // me encargo de la carga de más de 1 bloque:
1921             if(block != lastBlockLoaded)
1922             {
1923                 // TODO: podría necesitar chequeo de errores.
1924                 sd_read_block(imageSelected + block, dummyBufferTx, bufferRx);
1925                 lastBlockLoaded = block;
1926             }
1927
1928             bytesAvailable = 512 - offset;
1929             bytesTakenFromRow = (bytesNeeded < bytesAvailable) ? bytesNeeded : bytesAvailable; // cantidad de bytes que copio
1930
1931             memcpy(&(bufferEspectro[EspectroBufferIndex]), &(bufferRx[offset]), bytesTakenFromRow);
1932
1933             EspectroBufferIndex += bytesTakenFromRow / 2; // uint16 vs uint8
1934             bytesNeeded -= bytesTakenFromRow; // si me faltan, reitera
1935             block++;
1936             offset = 0;
1937         }
1938     }
1939 }
```

Tarea T_ILI9341, impresión del espectro total: Una de las directivas que acepta la tarea ILI9341 es la de procesar la información recibida desde el sensor espectral e imprimir sobre la diapositiva espectral el espectro resultante de la medición. Se tendrá que determinar el nivel de intensidad que hay sobre cada longitud de onda con una resolución de 1 nanómetro y luego se imprimirá verticalmente la intensidad relativa que irá de 0.0 a 1.0. Para la impresión se utilizará el buffer previamente cargado por la SD con el espectro color.

Función `ESPECTRO_algoritmoIntensidadEspectro()`: Aquí se confina la lógica necesaria para el análisis del espectro y su impresión en pantalla. Se realiza la toma de mediciones desde el sensor espectral, se verifica que se encuentre en rangos coherentes y se normaliza en base a la intensidad total medida. Luego se genera un vector flotante que abarca la totalidad del espectro visible y define un nivel de intensidad para cada longitud de onda con una resolución de 1 nanómetro en base a un análisis estadístico de la información recopilada. Una vez obtenido esto, se procede a mostrar en pantalla aquellos píxeles de intensidad correspondientes a la intensidad relativa medida en cada longitud de onda.

```
1846 void ESPECTRO_algoritmoIntensidadEspectro(void)
1847 {
1848     uint16_t r, g, b, c;
1849
1850     Adafruit_TCS34725_getRawData(&r, &g, &b, &c);
1851
1852     bool auxC1 = (c < r && c < g) ? true : false;
1853     bool auxC2 = (c < r && c < b) ? true : false;
1854     bool auxC3 = (c < g && c < b) ? true : false;
1855
1856     if(auxC1 || auxC2 || auxC3)
1857     {
1858         ILI9341.fillRect(20, 50, 280, 150, COLOR_MAGENTA);
1859         ILI9341.printText("SENSOR OVERFLOW", 25, 70, COLOR_BLACK, COLOR_MAGENTA, 2);
1860         ILI9341.printText("INTENTE NUEVAMENTE", 25, 100, COLOR_BLACK, COLOR_MAGENTA, 2);
1861         ILI9341.printText("ALEJESE DE LA FUENTE", 25, 130, COLOR_BLACK, COLOR_MAGENTA, 2);
1862         return;
1863     }
1864
1865     // Normalización y protección contra dividir por 0:
1866     float Rn = (c == 0) ? 0 : ((float)r / (float)c);
1867     float Gn = (c == 0) ? 0 : ((float)g / (float)c);
1868     float Bn = (c == 0) ? 0 : ((float)b / (float)c);
1869
1870     // chequeamos cuál tiene el valor mayor y lo usamos como referencia:
1871     float maxRGB = fmaxf(Rn, fmaxf(Gn, Bn));
1872     if(maxRGB > 1.0f)
1873     {
1874         Rn /= maxRGB;
1875         Gn /= maxRGB;
1876         Bn /= maxRGB;
1877     }
1878
1879     // toca mostrar en pantalla:
1880     for(uint32_t x = 0; x < 296; x++)
1881     {
1882         // defino mi longitud de onda en base a mi piso, mi ancho de banda y mi posición actual:
1883         float lambda = 380.0f + (float)x * (750.0f - 380.0f) / 295.0f;
1884         float I = intensity_from_wavelength(lambda, Rn, Gn, Bn);
1885         int32_t y = 130 - (int32_t)(I * 100.0f); // altura de intensidad de 100px
1886         intensity_I_measured[x] = I;
1887         while(y <= Espectro_y2)
1888         {
1889             uint16_t pixel = (((bufferEspectro[ (y-Espectro_y1) * 296 + x]) & 0xFF00) >> 8) | (((bufferEspectro[ (y-Espectro_y1) * 296 + x]) & 0x00FF) << 8);
1890             ILI9341.drawPixel(Espectro_x1 + x, y, pixel);
1891             y++;
1892         }
1893     }
1894 }
```

Subtarea *usb_task()*: Subrutina que ejecuta la tarea principal al alcanzar cierta instancia. Su tiempo de vida está limitado a cuando el usuario intenta enviar el espectro guardado a la PC. Su objetivo es identificar si llegó una petición de lectura desde PC mediante el periférico USB y de suceder eso, enviar por USB los datos guardados del espectro cuando la tarea principal se lo pida. La información total a enviar es de 296 flotantes provista por el sistema en una etapa previa, por lo que utilizando transferencias de 64 bytes se tiene que iterar el comando de envío hasta que se haya enviado la totalidad de la información.

```
*main.c X
2092 int8_t USB_task(void)
2093 {
2094     uint32_t sent_bytes = 0;
2095     if (flag_rx == 1)
2096     {
2097         //Check if the first byte of the report buffer equals 1
2098         if (report_buffer[0] == 1 || report_buffer[1] == 1)
2099         {
2100             //Turn the user LED on
2101             Adafruit_TCS34725_setInterrupt(false); // turn on LED
2102             //Turn the user LED off
2103             Adafruit_TCS34725_setInterrupt(true); // turn off LED
2104
2105             flag = 1; // me llegó de recepción el pedido de PC de enviar tramas.
2106         }
2107         flag_rx = 0;
2108         return 0;
2109     }
2110     //To send the output data when the button is pressed on PC:
2111     if (flag==1)
2112     {
2113         sent_bytes = 0;
2114         uint32_t float_index = 0;
2115         while(sent_bytes < (296 * sizeof(float)))
2116         {
2117             uint8_t USBD_state = USBD_CUSTOM_HID_SendReport(&hUsbDeviceFS, (uint8_t*)&(intensity_I_measured[float_index]), 64);
2118             //vTaskDelay(10);
2119             if(USBD_state == USBD_OK)
2120             {
2121                 sent_bytes += 64;
2122                 float_index += (64/sizeof(float));
2123             }
2124             if(USBD_state == USBD_BUSY)
2125                 continue;
2126             if(USBD_state == USBD_FAIL)
2127             {
2128                 return -1;
2129             }
2130         }
2131         flag = 0;
2132         return 0;
2133     }
2134     return 2;
2135 }
2136 }
```

Tarea T_detectar_pulsador(). Tarea de baja prioridad encargada de leer las entradas de los diferentes pulsadores que se utilizan para manejarse por el menú. Su uso es habilitado exclusivamente en ciertas instancias del menú. Su tarea es la de identificar si alguno de los pulsadores fue presionado y de haberlo sido, determinar si siguen pulsados después de un período corto de tiempo, momento en donde se envía a la tarea principal el botón que fue presionado. Luego se espera a que la tarea principal informe que la acción pulsada ya fue realizada y que la tarea pulsadores ya puede seguir leyendo las entradas nuevamente, no sin antes ratificar que el botón presionado previamente fue soltado.

```
1104 void T_detectar_pulsador(void* arg)
1105 {
1106     static userInteraction_t inputuser;
1107
1108     inputuser.actions = 0;
1109     inputuser.action_pending = 0;
1110     inputuser.pulsed = 0;
1111
1112     // espera inicialización:
1113     xSemaphoreTake(S_MDE_pulsador, portMAX_DELAY);
1114
1115     while(1)
1116     {
1117         if(inputuser.action_pending)
1118         {
1119             //xSemaphoreTake(S_MDE_pulsador, portMAX_DELAY);
1120             xQueueReceive(Q_pulsadoresEstado, &inputuser, portMAX_DELAY);
1121             continue;
1122         }
1123         if(inputuser.pulsed) // se presionó un botón y se tiene que soltar:
1124         {
1125             switch(inputuser.input_activated)
1126             {
1127                 case MOVE_UP:
1128                     if(HAL_GPIO_ReadPin(BOTON_UP_GPIO_Port, BOTON_UP_Pin) == GPIO_PIN_RESET)
1129                     {
1130                         inputuser.pulsed = false; // se soltó el botón
1131                         inputuser.input_activated = IDLE;
1132                     }
1133                     break;
1134                 case MOVE_DOWN:
1135                     if(HAL_GPIO_ReadPin(BOTON_DOWN_GPIO_Port, BOTON_DOWN_Pin) == GPIO_PIN_RESET)
1136                     {
1137                         inputuser.pulsed = false; // se soltó el botón
1138                         inputuser.input_activated = IDLE;
1139                     }
1140                     break;
1141                 case MOVE_LEFT:
1142                     if(HAL_GPIO_ReadPin(BOTON_LEFT_GPIO_Port, BOTON_LEFT_Pin) == GPIO_PIN_RESET)
1143                     {
1144                         inputuser.pulsed = false; // se soltó el botón
1145                         inputuser.input_activated = IDLE;
1146                     }
1147                     break;
1148                 case MOVE_RIGHT:
1149                     if(HAL_GPIO_ReadPin(BOTON_RIGHT_GPIO_Port, BOTON_RIGHT_Pin) == GPIO_PIN_RESET)
1150                     {
1151                         inputuser.pulsed = false; // se soltó el botón
1152                         inputuser.input_activated = IDLE;
1153                     }
1154                     break;
1155                 case SELECTION:
1156                     if(HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET)
1157                     {
1158                         inputuser.pulsed = false; // se soltó el botón
1159                         inputuser.input_activated = IDLE;
1160                     }
1161                     break;
1162                 default:
1163                     break;
1164             }
1165             continue;
1166         }
1167         switch(inputuser.inputs)
1168         {
1169             case MOVE_RIGHT:
1170                 // HAL_GPIO_ReadPin(BOTON_RIGHT_GPIO_Port, BOTON_RIGHT_Pin) == GPIO_PIN_SET
1171
1172                 xQueueSendToBack(Q_pulsadoresEstado, &inputuser, portMAX_DELAY);
1173                 xSemaphoreGive(S_pulsador);
1174             }
1175         }
1176     }
1177 }
```

● Porciones de código relevantes PC

Interfaz gráfica UI: La siguiente es una imagen de la aplicación en funcionamiento. Se dispone de 3 botones principales, uno para conectar la PC con el MCU, otro para pedir la información y guardarla en un archivo, y por último un botón de desconexión al terminar de utilizarla.



Botón Conectar: Al clicar el botón conectar se ejecuta el evento de botón clickeado, el cual se encarga de buscar entre todos los dispositivos USB conectados aquel que tenga el VID y PID específico de nuestro dispositivo, dispuesto arbitrariamente durante el desarrollo del proyecto. Una vez encontrado, se dispone a abrir una conexión con él y actualiza la interfaz gráfica.

```
43 void MainWindow::on_Conectar_BT_clicked()
44 {
45     signed int r;
46     unsigned int cnt;
47     unsigned int i;
48     signed int j, k, p;
49
50     connected = 1;
51     r = libusb_init(&ctx);
52     libusb_set_debug(ctx, 3); //set verbosity level to 3, as suggested in the documentation
53
54     cnt = libusb_get_device_list(ctx, &devs);
55
56     qDebug() << "Hay" << cnt << "dispositivos conectados";
57
58     for (i = 0; i < cnt; i++)
59     {
60         r = libusb_get_device_descriptor(devs[i], &desc);
61
62         if (r < 0)
63         {
64             qDebug() << "Error con el device descriptor del dispositivo";
65         }
66
67         if (desc.idProduct == MY_PRODUCT && desc.idVendor == MY_VENDOR)
68         {
69             Dispositivo = i;
70
71             qDebug() << "Mi dispositivo es el N°" << i;
72             qDebug() << "Tiene " << desc.bNumConfigurations << " cantidad de configuraciones";
73             qDebug() << "El Vendor ID es el N°: " << Qt::hex << desc.idVendor;
74             qDebug() << "El Product ID es el N°: " << Qt::hex << desc.idProduct;
75             qDebug() << "Device Class: " << desc.bDeviceClass;
76             qDebug() << "Serial Number: " << desc.iSerialNumber;
77
78             libusb_get_config_descriptor(devs[Dispositivo], 0, &config);
79
80             qDebug() << "Interfaces: " << (uint8_t) config->bNumInterfaces;
81             for (j=0; j<(uint8_t)(config->bNumInterfaces); j++)
82             {
83                 inter = &config->interface[j];
84                 qDebug() << "Cantidad de conf. alternativas: " << (uint8_t) (inter->num_altsetting);
85                 for (k=0; k < (uint8_t) (inter->num_altsetting); k++)
86                 {
87                     interdesc = &inter->altsetting[k];
88                     qDebug() << "Cant. de Interfaces: " << (uint8_t) interdesc->bInterfaceNumber;
89                     qDebug() << "Cant. de Endpoints: " << (uint8_t) interdesc->bNumEndpoints;
90                     for (p=0; p<(uint8_t) interdesc->bNumEndpoints; p++)
91                     {
92                         epdesc = &interdesc->endpoint[p];
93
94                         interdesc = &inter->altsetting[k];
95                         qDebug() << "Cant. de Interfaces: " << (uint8_t) interdesc->bInterfaceNumber;
96                         qDebug() << "Cant. de Endpoints: " << (uint8_t) interdesc->bNumEndpoints;
97                         for (p=0; p<(uint8_t) interdesc->bNumEndpoints; p++)
98                         {
99                             epdesc = &interdesc->endpoint[p];
100                             qDebug() << "Tipo de Descriptor: " << (uint8_t) epdesc->bDescriptorType;
101                             qDebug() << "Dirección de EP: " << Qt::hex << (uint8_t) epdesc->bEndpointAddress;
102                         }
103                     }
104                 }
105             }
106             libusb_free_config_descriptor(config);
107
108             r = libusb_open(devs[Dispositivo], &dev_handle);
109             if (r < 0)
110             {
111                 qDebug() << "Error" << r << "abriendo dispositivo";
112                 ui->box_text->setText("Error abriendo el dispositivo");
113                 return;
114             }
115             //libusb_set_auto_detach_kernel_driver(dev_handle, 1);
116             // Recommended "Safe" Manual Detach
117             if (libusb_kernel_driver_active(dev_handle, 0) == 1)
118             {
119                 qDebug() << "driver active returned 1, detaching...";
120                 int g = libusb_detach_kernel_driver(dev_handle, 0);
121                 if (g != 0)
122                 {
123                     // Handle error only if it's not "already detached"
124                     qDebug() << "not already detached, error?";
125                 }
126             }
127             else
128             {
129                 qDebug() << "creo que salio bien el detach";
130             }
131             ui->Conectar_BT->setEnabled(false);
132             ui->Enviar_BT->setEnabled(true);
133             ui->desconectar->setEnabled(true);
134             ui->box_text->setText("Conectado");
135             return;
136         }
137     }
138     ui->box_text->setText("No se encontró el dispositivo");
139 }
```

Botón Pedir Info: Al clicar el botón Pedir Info se ejecuta el evento de botón clickeado, el cual inicia la búsqueda de información desde el MCU y de ser exitosa, guarda el contenido en un archivo .csv con el nombre de la fecha y hora actuales preguntándole al usuario dónde quiere guardarlo. Este botón debe ser clickeado luego de seleccionar en el dispositivo la opción de subir datos a PC por USB e inicie el *timeout*.

```
155
156 void MainWindow::on_Enviar_Bt_clicked()
157 {
158
159     std::vector<float> v;
160     const int expectedCount = 296;
161     bool ok = requestAndReadFloatVector(v, expectedCount, 500);
162
163     int actual_length;
164     unsigned int Enviados;
165
166     if(!ok)
167     {
168         qDebug() << "Error leyendo vector desde el dispositivo";
169         return;
170     }
171     else
172     {
173         qDebug() << "lectura completada, creando csv...";
174     }
175     //Pedir nombre de archivo:
176     QString defaultName = QDateTime::currentDateTime().toString("yyyyMMdd_HHmss") + "_intensity.csv";
177     QString path = QFileDialog::getSaveFileName(this, "Guardar CSV", defaultName, "CSV files (*.csv)");
178     if(path.isEmpty())
179     {
180         qDebug() << "Lectura completada, guardado cancelado por el usuario.";
181         return;
182     }
183     QFile f(path);
184     if(!f.open(QIODevice::WriteOnly | QIODevice::Text))
185     {
186         QMessageBox::critical(this, "Error", "No se pudo abrir el archivo para escribir.");
187         qDebug() << "Error guardando CSV.";
188         return;
189     }
190     QTextStream out(&f);
191     out << "index,value\n";
192     for(int i = 0; i < (int)v.size(); ++i)
193     {
194         out << i << ", " << QString::number(v[i], 'g', 9) << "\n";
195     }
196     f.close();
197
198     //labelStatus->setText(QString("Vector guardado (%1 valores) en: \n%2").arg(v.size()).arg(path));
199     qDebug() << "VEctor guardado (" << v.size() << " valores) en : \n" << path;
200     ui->box_text->setText("Datos recibidos y guardados");
201 }
```

Función `requestAndReadFloatVector()`. Esta función ejecutada al clicar Pedir Info se encarga de notificar al MCU que quiero recibir la información guardada del espectro y procede a recibir múltiples paquetes hasta haber guardado el espectro completo. Luego, antes de devolver la información recibida, trabaja los datos para volver a recomponer los valores flotantes en base a los bytes recibidos.

```
215 bool MainWindow::requestAndReadFloatVector(std::vector<float> &out, int expectedCount, int timeoutMs)
216 {
217     // protocolo simple:
218     // escribimos un informe (report id = 0) con el primer byte = 0x01 (comando -> pedir vector)
219     // el dispositivo respondera con uno o varios reportes que contienen los bytes del vector (little
220     // acumulamos hasta expectedCount * 4 bytes o timeout
221
222     const int reportSize = 64; // tamaño típico; hid_read leera hasta esto
223     int neededBytes = expectedCount * 4;
224     int actual_length;
225     signed int Enviados;
226     std::vector<uint8_t> bufferAll;
227     bufferAll.reserve(neededBytes);
228
229     // send request:
230     uint8_t req[64];
231     memset(req, 0, sizeof(req));
232     // en hidapi en muchas plataformas, req[0] es report-id 0 si se usa ninguno
233     req[0] = 0x01; // comando 1 -> pedir vector (esto debe coincidir con firmware)
234     /* ---- ajustar al uso debido ---- */
235
236     // 3. CHECK the current configuration before setting it
237     int current_config = 99;
238     libusb_get_configuration(dev_handle, &current_config);
239     qDebug() << "current config is " << current_config;
240     if (current_config != 1)
241     {
242         int u = libusb_set_configuration(dev_handle, 1);
243         if (u != 0)
244         {
245             // If it still fails with errno 32, the device might already be
246             // ready enough to just proceed to claiming.
247         }
248     }
249
250     libusb_claim_interface(dev_handle, 0);
251
252     Enviados = libusb_interrupt_transfer(dev_handle, 0x01, req, 64, &actual_length, timeoutMs);
253     //libusb_interrupt_transfer(dev_handle, 0x01, RxData, sizeof(RxData), &actual_length, 0);
254
255     //int res = hid_write(deviceHandle_, req, 65);
256     if (Enviados < 0)
257     {
258         qWarning("write_error");
259         return false;
260     }
261     else
262     {
263         qDebug() << "recibi del stm " << actual_length << " bytes";
264     }
265
266     // ahora leemos repetidamente hasta obtener los bytes:
267     QElapsedTimer timer;
268     timer.start();
269     // usamos bloqueos con timeout parcial
270     qDebug() << "inicio timer";
271     while((int)bufferAll.size() < neededBytes && timer.elapsed() < timeoutMs)
272     {
273         // hid_read bloquea; preferimos hid_read_timeout si está disponible
274         // hid_read_timeout de hidapi puede usarse, pero no siempre está expuesto en algunos builds.
275         // intentaremos hid_read with a small blocking read and bail out con timeout global.
276
277         if (Enviados < 0)
278         {
279             qWarning("write_error");
280             return false;
281         }
282         else if (actual_length == 0)
283         {
284             // sin datos inmediatos; permitir que el loop vuelva a intentar hasta timeout
285             QThread::msleep(5);
286             qDebug() << "durmiento";
287             continue;
288         }
289         else
290         {
291             // agregamos got bytes
292             bufferAll.insert(bufferAll.end(), req, req + actual_length);
293             qDebug() << "inserto bytes";
294         }
295     }
296     if ((int)bufferAll.size() < neededBytes)
297     {
298         qWarning("No se recibieron suficientes bytes: %d / %d", (int)bufferAll.size(), neededBytes);
299         return false;
300     }
301     qDebug() << "transformando lo recibido en float:";
302     // parse floats little-endian
303     out.resize(expectedCount);
304     for (int i = 0; i < expectedCount; ++i)
305     {
306         const uint8_t *p = &bufferAll[i*4];
307         float f = readFloatLE(p);
308         out[i] = f;
309     }
310     qDebug() << "transformado listo";
311     return true;
312 }
```

Botón Desconectar. Al clicar el botón Desconectar se ejecuta el evento de botón clickeado, el cual simplemente cierra la conexión por USB con el MCU para evitar dejar sockets abiertos.

```
320 void MainWindow::on_desconectar_clicked()
321 {
322     if (dev_handle)
323     {
324         // 1. Release the interface you claimed (usually 0)
325         libusb_release_interface(dev_handle, 0);
326
327         // 2. Re-attach kernel driver (optional, Linux specific)
328         // If you detached the driver to gain control, this gives it back to the OS
329         libusb_attach_kernel_driver(dev_handle, 0);
330
331         // 3. Close the device handle
332         libusb_close(dev_handle);
333         dev_handle = nullptr; // Prevent double-closing
334     }
335
336     // 4. Update your UI (Disable the disconnect button, enable connect button)
337     ui->box_text->setText("Desconectado");
338     ui->Conectar_Bt->setEnabled(true);
339     ui->Enviar_Bt->setEnabled(false);
340     ui->desconectar->setEnabled(false);
341 }
342
```

● Consumo Eléctrico:

Al ser un proyecto alimentado por pilas (portátil) y no por una fuente regulada (enchufable a la red eléctrica), se midió el consumo de corriente y nos dio un valor de 150-200 [mA].

Usando 4 pilas de 1.5V de 2000 mAh nos da una autonomía aproximada de 10 horas, pero hay que tener en cuenta que el voltaje suministrado debe ser mayor a 5.30 [V].

$$6V \times t = 5.3V$$

$$t = 0.88.$$

Las pilas deben usarse en entre un 10% y 12%, con un consumo de 200 mA podemos asegurar una autonomía de 1 Hora 15 minutos aproximadamente teniendo en cuenta que las pilas pueden suministrar más de 6V en el peor de los casos.

● Dispositivos y periféricos empleados, detalles

- Pantalla TFT 2.4" ILI9341
- Memoria SD de 16GB
- Sensor LDR
- Sensor RGB TCS34725
- Tact Switch's 6mm x 6mm
- Mini USB tipo B
- Placa PCB diseñada para el mismo

● Tipos de comunicación empleadas

Se usaron 3 tipos de comunicaciones:

SPI la cual se empleó para la comunicación de la pantalla TFT y la SD.

I2C para la comunicación del módulo TCS34725

USB para la comunicación con la PC para transmitir la información obtenida y almacenarla en un archivo.

● Cronogramas del proyecto

Actividad	Responsable	Fecha de Inicio	Fecha Final	Estado
Realización de Idea Fuerza	Todos	11-05-2025	12-05-2025	Completada
Entrega de Idea Fuerza	Todos	14-05-2025	14-05-2025	Completada
Realización de Anteproyecto	Todos	10-06-2025	12-06-2025	Completada
Entrega de Anteproyecto	Todos	14-06-2025	14-06-2025	Completada
Primera compra de componentes	Franco Mendez	15-08-2025	15-08-2025	Completada
Programación de Pantalla TFT	Martin Silvosa	16-10-2025	25-10-2025	Completada
Diseño de la Placa	Franco Mendez	17-10-2025	24-10-2025	Completada
Programación de ADC	Martin Silvosa	26-10-2025	28-10-2025	Completada
Programacion sensor RGB	Martin Silvosa	1-11-2025	6-11-2025	Completada
Segunda compra de componentes	Franco Mendez	10-11-2025	11-11-2025	Completada
Fabricación de la Placa	Franco Mendez	12-11-2025	13-11-2025	Completada
Diseño de Maqueta 3D	Franco Mendez	14-11-2025	16-11-2025	Completada
Fabricación de Maqueta 3D	Franco Mendez	17-11-2025	17-11-2025	Completada
Programación QT	Martin Silvosa	20-11-2025	23-11-2025	Completada
Ensamble y testeo de Placa	Todos	1-12-2025	1-12-2025	Completada
Arreglo de Problemas	Todos	2-12-2025	14-12-2025	Completada
Segundo Testeo del Proyecto	Todos	16-12-2025	16-12-2025	Completada
Arreglo de Problemas	Todos	16-12-2025	22-12-2025	Completada
Entrega Final	Todos	23-12-2025	23-12-2025	Completada

- Costos del proyecto

Productos	Precio [ARG]
Pantalla TFT 2.4"	\$18.600
Sensor LDR	\$2.053
Sensor RGB TCS34725	\$13.341
Portapilas	\$1.611
Pilas	\$6.500
Impresión 3D	\$6000
NúcleoF446RE	\$24.690
Placa de cobre	\$1.969
USB	\$1.733
Borneras, botones y elementos Pasivos	\$1200
Subtotal:	\$77.697

Teniendo en cuenta la participación de dos Profesionales Juniors y un estimado de 50 Hs se trabajo y según la [Copitec](#).

Subtotal	\$77.697
Mano de obra	\$9.250.000
Total	\$9.327.697

Haciendo la sumatoria de todos los costos termina dando un valor todo el proyecto

● Desafíos

Diseño:

A la hora de plantear la interfaz gráfica que se le mostraría al usuario, habíamos empezado con una resolución menor a la actual, pero rápidamente nos dimos cuenta que no sería suficiente para mostrar la información necesaria que queríamos visualizar, por lo que triplicamos la resolución superficial y eso conllevó a que las diapositivas necesarias para el menú gráfico fueran muy pesadas en términos de memoria FLASH y ya no cabrían sobre la núcleo, necesitando así una memoria externa y optando por utilizar la ranura SD ya integrada por el TFT ILI9341.

Ensamblaje:

Una de las problemáticas encontradas fue que la placa fue diseñada para usar tanto la memoria SD como la pantalla por el mismo puerto SPI, pero todo el código estaba diseñado para que funcione en distintos SPI, por lo que se optó por usar cables dupont y conectarlos directamente a la Núcleo en vez de rediseñar la placa.

Otra problemática fue que al alimentar la núcleo de forma externa, éste no iniciaba por sí solo tras energizar la placa, necesitando apretar el botón de RESET para que inicie normalmente. Se optó por colocar un RC de 330k/100uF entre los pines NRST y GND. Esto logró que se realizara un reinicio automático 4 segundos después del energizado del dispositivo y que el proyecto funcione con normalidad. Se trató de elegir diferentes valores de capacitores pero no se obtuvo un reinicio más rápido, sin embargo valores más grandes lograban un reinicio más lento.

● Conclusiones

Fuimos capaces de integrar tanto los conocimientos teóricos como los prácticos en el transcurso de todo el proyecto como puede ser el uso del microcontrolador y programación en sistemas embebidos en Freertos, habilidades para diseño de PCBs y soldadura, como también diseño de la maqueta en SCAD.

Este proyecto nos permitió integrar y aplicar todo lo que hemos aprendido a lo largo de la carrera. Es emocionante ver cómo los conceptos teóricos se vuelven tangibles y funcionales, lo que nos da confianza en nuestras habilidades como futuros ingenieros. Este tipo de proyectos no sólo consolida lo aprendido, sino que también nos acerca a la realidad profesional, demostrando que contamos con las habilidades necesarias para enfrentar desafíos técnicos concretos y desarrollar soluciones creativas y eficaces.

● Bibliografía

- Configuración de la SD para trabajarla bajo el STM en modo SPI:
https://www.st.com/resource/en/application_note/an5595-spc58xexspc58xgx-multimedia-card-via-spi-interface-stmicroelectronics.pdf
https://elm-chan.org/docs/mmc/mmc_e.html
<https://yaseen.ly/writing-data-to-sdcards-without-a-filesystem-spi/>
- Configuración de la pantalla TFT ILI9341:
<https://cdn-shop.adafruit.com/datasheets/ILI9341.pdf>
https://www.lcdwiki.com/res/MSP2402/2.4inch_SPI_Module_MSP2402_User_Manual_EN.pdf
- Configuración del sensor espectral TCS34725:
<https://cdn-shop.adafruit.com/datasheets/TCS34725.pdf>

● Panel de firmas

	
Integrante I	Integrante II
Fecha: 23/12/2025	Fecha: 23/12/2025